

Proyecto Final – Multipagos QR

1) Caso de estudio (voz del cliente)

*“Somos una plataforma boliviana que quiere centralizar **pagos de servicios** (agua, luz, internet, colegios, etc.).*

*Nuestros usuarios deberían poder **pagar sin crear cuenta**: entran, escriben su **CI/NIT o código de cliente**, el sistema trae su **deuda**, generamos un **QR** y, si el pago se **aprueba**, emitimos un **comprobante**.*

*Además, trabajamos con **empresas proveedoras** (por ejemplo, la cooperativa de agua o la eléctrica). Cada empresa necesita **subir sus deudas** (idealmente por **CSV**) y consultar **los pagos que recibió**, con filtros y exportación.*

*Como administradores de la plataforma, debemos **dar de alta empresas**, habilitar sus **servicios** (definir qué campos piden para identificar al cliente), **monitorear transacciones** y ver **reportes**.*

*Es clave que la plataforma sea **multi-empresa (multi-tenant)**: cada empresa ve solo sus datos. No necesitamos integraciones complejas (ni webhooks reales); los **pagos y QR** pueden ser **simulados** para efectos académicos.”*

2) Lista de funcionalidades

Público (sin autenticación; cuenta opcional)

- **Búsqueda de deuda por identificador** (CI/NIT/código de cliente + proveedor/servicio).
- **Detalle de deuda**: monto, periodo, vencimiento (simulado), recargos (opcionales).
- **Generación de QR y pago simulado** (aprobado/rechazado).
- **Comprobante** descargable (PDF/HTML) y **consulta** de estado de la transacción.
- **(Opcional)** Crear cuenta para **guardar identificadores y ver historial**.

Proveedor (empresa afiliada)

- **Ingreso al panel** de su empresa (multi-tenant).
- **Carga de deudas por CSV** y edición puntual (identificador, periodo, monto, estado).
- **Definición de campos requeridos** por servicio (p.ej., “Nro. Suministro”).
- **Listado de pagos recibidos con filtros** (fecha, servicio, estado) y **exportación** (CSV/Excel).
- **Cambio de estado** de registros relevantes (por ejemplo, marcado interno de conciliación básica).

Administrador de Multipagos

- **Alta/edición/aprobación de empresas** (logo, datos fiscales) y **servicios** por empresa.
- **Catálogo público** de proveedores/servicios visibles.
- **Monitoreo global** de transacciones con filtros.
- **Reportes** (volumen por empresa/servicio, tasa de aprobación, etc.) y **descargas**.

Requerimientos transversales

- **Multi-empresa real:** toda consulta protegida filtra por `company_id` asociado al usuario.
 - **Seguridad:** autenticación (cuando aplique), autorización por rol, hashing de contraseñas, validaciones
 - **UX responsivo** (web y móvil).
 - **Datos de prueba:** ≥ 2 empresas, servicios y deudas variadas.
-

3) Propuesta de microservicios

Cada servicio con **BD propia** y **APIs REST**. Comunicación sincrónica (REST). **Sin webhooks**. El pago/QR es **simulado**.

(1) Auth & Tenancy

- **Responsabilidad:** usuarios, login (JWT), **roles** (`public` , `user` , `provider` , `admin`), **empresas** (multi-tenant) y asignación usuario $\leftrightarrow$ empresa.
- **Modelos clave:** `users` , `companies` , `user_companies(role)` .
- **Por qué existe:** base de seguridad y aislamiento de datos por empresa.

(2) Proveedores & Catálogo

- **Responsabilidad:** lifecycle de **empresas** y **servicios** por empresa; definición de **campos requeridos** para identificar deudas.
- **Modelos clave:** `tenants/companies` , `services(input_schema_json, active)` .
- **APIs:** alta/aprobación de empresa, CRUD de servicios, endpoint público de catálogo.
- **Por qué existe:** expone qué se puede pagar y cómo identificar al cliente.

(3) Deudas (Billing)

- **Responsabilidad:** **carga** (CSV) y **gestión** de deudas; **lookup** por identificador; cambio de estado (`PENDING/PAID/CANCELLED`).
- **Modelos clave:** `debts(tenant_id, service_id, customer_ref, period, amount, due_date, status)` , `imports` .
- **APIs:** `POST /debts/import` , `POST /debts/lookup` , `PATCH /debts/{id}` .
- **Por qué existe:** es el "inventario" de lo que se debe.

(4) Pagos & QR

- **Responsabilidad:** **generar QR**, **simular cobro**, registrar **transacciones** y **actualizar** la deuda a `PAID` en éxito; manejar **recibos** simples.
- **Modelos clave:** `transactions(tenant_id, service_id, customer_ref, amount, status, receipt_hash)` .
- **APIs:** `POST /payments/qr` , `POST /payments/confirm` , `GET /payments?filters` .
- **Por qué existe:** encapsula el proceso de pago y su estado, sin integrar pasarelas reales.

(5) Reportes & Administración

- **Responsabilidad:** vistas **agregadas** (KPIs por empresa/servicio), **exportaciones** (CSV/Excel), **auditoría** básica y tablero admin.
- **Modelos clave:** `audit_logs` , tablas/vistas de agregación.
- **APIs:** reportes por empresa/global; consulta de bitácora; endpoints de administración global.
- **Por qué existe:** visibilidad y control de la operación sin mezclar con la lógica de cobro o deuda.